



RAG-Labs — Environment & Data Setup Guide

This guide gets you from a fresh machine to a running RAG lab notebook, on **Windows, macOS, or Linux**, and tells you exactly where each lab's data has to live. It applies to **Labs 1–10** (each has a notebook). Lab 11, the capstone, is a design spec that reuses the earlier labs, so it needs no setup of its own. Do the setup **once per lab**: each lab has its own virtual environment and its own `.env` file. Pick **one** of the three setup paths below; you don't need all three.

Before you start, you need:

- **Python 3.10 or newer.** If you're not sure whether you have it, see [Installing Python](#).
- A **free OpenRouter account** and an API key that starts with `sk-or-v1-`. Sign up at [openrouter.ai](#) → Keys → Create Key. Every lab uses a free-tier model, so the LLM costs nothing.
- **Only for some labs:** a free **Neo4j Aura** instance (Labs 7, 8, 10), a free **Qdrant Cloud** cluster (Labs 3, 4) or a **PageIndex** API key (Lab 9). See [Cloud services](#); set up only what the labs you plan to run need.
- **Disk space and time.** Plan for roughly 1–3 GB per lab for labs that install PyTorch and sentence-transformers (1, 2, 5, 8, 10), and much less for the others. The first run also downloads small embedding or OCR models, so it needs internet and a few extra minutes.
- **Google Colab needs none of the local install** — skip straight to Option 2 if you'd rather not install anything locally.

What each lab needs at a glance

Use this table to see what to prepare before opening a notebook. The “Data” column is explained in full in [Data Setup and Loading](#).

Lab	Services (besides OpenRouter)	Data the notebook reads	Where the data must be
1. Classical RAG	—	<code>sample_text_document.txt</code>	<code>data/</code> in the lab folder — ships with the repo
2. Hybrid RAG	—	<code>sample_text_document.pdf</code>	<code>data/</code> in the lab folder — ships with the repo
3. Parent-Child	Qdrant Cloud (or in-memory option)	<code>attention_paper.pdf</code>	Directly in the lab folder , beside the notebook — downloaded by the notebook
4. CoBERT	Qdrant Cloud	<code>bert_paper.pdf</code>	Directly in the lab folder , beside the notebook — downloaded by the notebook

Lab	Services (besides OpenRouter)	Data the notebook reads	Where the data must be
5. OCR + RAG	—	<code>debate.pdf</code> and 3 invoice images	<code>data/</code> and <code>data/invoices/</code> — downloaded by the notebook (copies ship with the repo)
6. LLM Wiki	—	<code>SunFactSheet.pdf</code>	<code>data/</code> — downloaded by the notebook (a copy ships with the repo, used if the download fails)
7. Graph RAG	Neo4j Aura	<code>document.pdf</code>	<code>data/</code> — downloaded by the notebook
8. Graph + Vector	Neo4j Aura	<code>downloaded_paper.pdf</code>	<code>data/</code> — downloaded by the notebook
9. Vectorless RAG	PageIndex	<code>CCS-Q1-2025-Earnings-Release.pdf</code>	<code>data/</code> — must already exist ; the notebook does not download it
10. Agentic RAG	Neo4j Aura	<code>downloaded_paper.pdf</code>	<code>data/</code> — downloaded by the notebook

Installing Python

Skip this section if `python3 --version` (macOS/Linux) or `python --version` (Windows) already prints `3.10` or higher.

Windows

1. Download the installer from python.org/downloads (the “Windows installer (64-bit)” link).
2. Run it, and **check the box “Add python.exe to PATH”** at the bottom of the first screen. This is the step almost everyone misses, and without it none of the commands below will work.
3. Click **Install Now**.
4. Open a new PowerShell or Command Prompt window and confirm: `python --version`.

Alternative: install from the Microsoft Store (search “Python 3.12”) — it handles PATH automatically.

macOS

The system Python on macOS is old and shouldn't be used for these labs. Two good options:

- **Homebrew** (recommended if you don't already manage Python another way): install Homebrew from brew.sh, then run `brew install python@3.12`.
- **Official installer**: download the macOS installer from python.org/downloads and run it.

Confirm with: `python3 --version`.

Linux

Most distributions ship a recent-enough Python. Confirm with `python3 --version`; if it's below 3.10, install a newer one with your package manager, e.g.:

```
sudo apt update && sudo apt install python3 python3-venv python3-pip # Debian/Ubuntu
sudo dnf install python3 python3-pip # Fedora
```

Installing JupyterLab

You only need this for **Option 1 (Jupyter Notebook)** or **Option 3 (VS Code)**. Google Colab already has Jupyter running in your browser. Each lab's notebook starts with its own `!pip install` cell for the packages that lab needs; add `jupyterlab` and `ipykernel` to that command if they are not already installed in your virtual environment.

To install it manually (macOS/Linux shown; on Windows use the create/activate commands from the table in Option 1, step 2):

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install jupyterlab ipykernel
```

Verify with `jupyter --version` — it should print a version table including `jupyterlab`. If you get “command not found” or “jupyter’ is not recognized”, your virtual environment likely isn’t activated; see the Troubleshooting table.

Choose Your Environment

	Jupyter Notebook	Google Colab	VS Code
Runs on	Your own machine	Google's servers (browser only)	Your own machine
Works on	Windows, macOS, Linux	Any OS with a browser	Windows, macOS, Linux
Install required?	Yes (Python + packages)	No — just a Google account	Yes (VS Code + extensions)
Best for	Working fully offline once set up	Quickest start, no local install	Students who already code in VS Code
.env handling	Native — a real file in the lab folder	Needs an extra step (no persistent disk)	Native — a real file in the lab folder
Data files	Already in the cloned repo	Must be uploaded or cloned each session	Already in the cloned repo
Setup time	~10 min	~3 min	~10 min

If you're not sure, use **Jupyter Notebook** — it's the path every lab's own instructions assume.

Get the Repository First

The labs live in one repository, `RAG-Labs`, with one folder per section and one numbered folder per lab. **Get the whole repository**, not just the notebook, because several labs read files that ship with it (see [Data Setup](#)).

```
git clone <your-repo-url> RAG-Labs      # or download the ZIP and extract it
cd RAG-Labs
ls                                       # section-1-classical-rag, section-2-...,
README.md
```

Each lab folder holds everything for that lab:

```
section-1-classical-rag/
  lab-01-classical-rag/
    lab_classical_rag.ipynb    <- the notebook you run
    lab_classical_rag.md       <- the written walkthrough
    data/                     <- input files (when the lab uses them)
    .env                      <- YOU create this (keys); never committed
```

Option 1: Jupyter Notebook (Local)

1. Open a terminal and move into the lab folder. Go to the specific lab folder, not the repository root. The notebooks use relative paths (`".env"`, `"data/..."`), so they only work when Jupyter is started from inside the lab folder. The commands are the same on Windows, macOS and Linux:

```
cd RAG-Labs/section-1-classical-rag/lab-01-classical-rag  # replace with the lab you're
doing
```

2. Create and activate a virtual environment. This keeps each lab's packages isolated. The **create** command is the same everywhere; the **activate** command differs by platform and shell:

```
python3 -m venv .env    # macOS/Linux
python -m venv .env     # Windows
```

Platform / shell	Activate command
macOS / Linux (bash, zsh)	<code>source .env/bin/activate</code>
Windows — Command Prompt	<code>.env\Scripts\activate.bat</code>
Windows — PowerShell	<code>.env\Scripts\Activate.ps1</code>
Windows — Git Bash	<code>source .env/Scripts/activate</code>

Your terminal prompt should now show `(.env)` at the start of the line. (PowerShell note: if you get a “running scripts is disabled” error, run `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` once, then retry.)

3. Install the packages. Each notebook's first code cell is a `!pip install` line with exactly the packages that lab needs, and the lab's own "Environment / Dependencies Setup" section repeats it. Use that cell, or copy the line into the terminal without the leading `!`. The RAG labs do not pin exact versions, so install the packages the lab names. Lab 9 also ships a `requirements.txt`:

```
pip install --upgrade pip
pip install jupyterlab ipykernel
pip install -r requirements.txt      # Lab 9 only; other labs use their notebook's pip cell
```

4. Set up your `.env` file. Create it inside the lab folder, next to the `.ipynb` file, using the keys in the "What each lab needs" table (details in the [.env section](#)). The RAG repo does not include a `.env.example` template, so create the file yourself:

```
# macOS / Linux / Git Bash
printf 'OPENROUTER_API_KEY=sk-or-v1-your-real-key-here\n' > .env
# Windows PowerShell
Set-Content .env 'OPENROUTER_API_KEY=sk-or-v1-your-real-key-here'
# Windows Command Prompt
echo OPENROUTER_API_KEY=sk-or-v1-your-real-key-here> .env
```

5. Check that the data is where the notebook expects it. From the lab folder, with the environment active:

```
python -c "from pathlib import Path; print(Path.cwd()); print(sorted(p.name for p in Path('data').glob('*'))) if Path('data').exists() else 'no data/ folder!'"
```

For Labs 1, 2, 5, 6, 9 you should see the files listed in the data table. Labs 3, 4, 7, 8 and 10 create or fill their files on the first run, so an empty or missing `data/` folder is fine there.

6. Launch Jupyter and open the notebook.

```
jupyter lab lab_classical_rag.ipynb      # replace with this lab's .ipynb filename
```

Run each cell top to bottom with `Shift+Enter`. Don't skip ahead — later cells depend on variables created earlier. **7. When you're done**, run `deactivate` (optional, but tidy).

Option 2: Google Colab

Colab runs in your browser on Google's servers, so there is nothing to install, but it has **no persistent disk by default**. That affects your packages, your API key **and your data files**.

1. Open the notebook. Go to colab.research.google.com → File → Upload notebook, and select the lab's `.ipynb` file. (Or File → Open notebook → GitHub tab, if the repository is public.)

2. Install packages in a cell. Run the notebook's own first cell, the one that starts with `!pip install`. Colab keeps some preinstalled packages, so a restart prompt can appear; if it does, restart the runtime and continue from the next cell.

3. Provide your API keys the secure way. Do **not** paste keys into a code cell. Use Colab Secrets: click the **key icon** in the left sidebar → Add new secret → name it `OPENROUTER_API_KEY` → paste the value →

toggle **Notebook access** on. Add one secret per key the lab needs (`NEO4J_URI` , `NEO4J_USERNAME` , `NEO4J_PASSWORD` , `QDRANT_URL` , `QDRANT_API_KEY` , `PAGEINDEX_API_KEY`). Then load them before the lab's own `load_dotenv()` cell:

```
import os
from google.colab import userdata
for k in ["OPENROUTER_API_KEY", "NEO4J_URI", "NEO4J_USERNAME", "NEO4J_PASSWORD",
         "QDRANT_URL", "QDRANT_API_KEY", "PAGEINDEX_API_KEY"]:
    try:
        os.environ[k] = userdata.get(k)
    except Exception:
        pass          # this lab does not use that key
```

4. Alternative: write a real `.env`. Run `%%writefile .env` in a cell with the `KEY=value` lines. It is wiped whenever the runtime disconnects, so Secrets is the better default.

5. Get the data into Colab. Colab starts with an empty disk, so choose one:

- **Clone the repository (recommended)** and move into the lab folder, so the notebook's relative paths work unchanged:

```
!git clone <your-repo-url> RAG-Labs
%cd RAG-Labs/section-1-classical-rag/lab-01-classical-rag
```

- **Or upload just the files:** open the Files panel (folder icon), create a folder named `data` and upload the lab's file(s) into it (Labs 1, 2, 5, 6, 9). For Labs 3 and 4, upload nothing: the PDF is downloaded next to the notebook.

The Colab disk is deleted when the runtime is recycled, so repeat step 5 (and the Secrets step is already saved) at the start of each new session. **6. Run the notebook** top to bottom with `Shift+Enter`.

Option 3: VS Code

VS Code is identical across Windows, macOS and Linux; the only differences are the same `venv`-activation and file-creation commands shown in Option 1.

1. **Install two extensions** (Ctrl+Shift+X on Windows/Linux, Cmd+Shift+X on macOS): **Python** and **Jupyter**, both published by Microsoft.
2. **Open the lab folder.** File → Open Folder... and select the specific lab folder (e.g. `lab-01-classical-rag`), not the whole repository. This keeps the Python interpreter picker scoped correctly and makes the notebook's relative paths work.
3. **Create a virtual environment.** In the built-in terminal (Ctrl+`) run the create/activate commands from Option 1, step 2. Or use the Command Palette → “Python: Create Environment” → Venv.
4. **Install the packages** as in Option 1, step 3.
5. **Select the interpreter.** Command Palette → “Python: Select Interpreter” → choose the `.venv` you created. This is the step people miss most often; with the wrong interpreter, imports fail even though you installed the package.

6. **Create your `.env`** . Explorer panel → New File → name it exactly `.env` → type the key lines → save.
This works the same on every platform.
7. **Open the `.ipynb` directly in VS Code**, click the kernel picker in the top right to confirm it uses your `.venv` , then run cells with `Shift+Enter` .

Creating and Configuring Your `.env` File

What it is and why it matters

A `.env` file is a small plain-text file that holds your secrets outside your code. Every RAG notebook calls `load_dotenv(".env")` at the start, which reads this file and puts its values into `os.environ` . The code then reads them with `os.getenv("OPENROUTER_API_KEY")` instead of having a key typed into a cell. This matters because **code gets shared and committed to git; secrets shouldn't be**.

One file per lab, in the lab folder

The path is relative, so the file must sit **next to the** `.ipynb` you run. A `.env` in the repository root is **not** picked up. Contents depend on the lab:

```
# Labs 1, 2, 5, 6 -- OpenRouter only
OPENROUTER_API_KEY=sk-or-v1-your-real-key-here

# Labs 3, 4 -- add Qdrant
OPENROUTER_API_KEY=sk-or-v1-your-real-key-here
QDRANT_URL=https://your-cluster-id.eu-central.aws.cloud.qdrant.io:6333
QDRANT_API_KEY=your-qdrant-key

# Labs 7, 8, 10 -- add Neo4j
OPENROUTER_API_KEY=sk-or-v1-your-real-key-here
NEO4J_URI=neo4j+s://xxxxxxx.databases.neo4j.io
NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=your-generated-password
NEO4J_DATABASE=neo4j

# Lab 9 -- add PageIndex
OPENROUTER_API_KEY=sk-or-v1-your-real-key-here
PAGEINDEX_API_KEY=your-pageindex-key
```

Ways to create the file by hand:

Where	How
macOS/Linux terminal	<code>nano .env</code> (or <code>vim .env</code>), type the lines, save and exit
Windows — PowerShell	<code>New-Item .env</code> then <code>notepad .env</code>
Windows — Command Prompt	<code>type nul > .env</code> then <code>notepad .env</code>
Windows — Notepad directly	File → Save As → filename <code>.env</code> , and change “Save as type” to All Files (otherwise Notepad silently saves <code>.env.txt</code>)

Where	How
VS Code (any platform)	Explorer panel → New File → name it <code>.env</code> → type the lines → save
Google Colab	Use Colab Secrets, or <code>%%writefile .env</code> in a cell (see Option 2)

A few details that trip people up, on any platform:

- **No quotes needed** around values, and **no spaces around the** `=`: `KEY=value`, not `KEY = value`.
- **The filename is exactly** `.env` — a leading dot and no extension. Check it in your file browser with file extensions turned on.
- **Neo4j**: keep the `neo4j+s://` prefix on `NEO4J_URI`. The **s** enables the encrypted connection Aura requires.
- **Qdrant**: keep the `:6333` port at the end of `QDRANT_URL`. Without it the client cannot connect.
- **Save, then re-run the** `load_dotenv()` **cell**. Values are loaded once, when that cell runs, so after editing the file re-run the cell (or restart the kernel).

Keeping it safe

- **Never commit** `.env` **to git**. Each lab's `.gitignore` excludes it, but double-check with `git status` before pushing.
- **Never paste a real key into a notebook cell, a chat, or a screenshot.**
- **If a key leaks**, revoke it at once in that service's dashboard (for OpenRouter: openrouter.ai/keys) and create a new one.

Cloud Services (only for the labs that need them)

OpenRouter API key — every lab

Sign up at openrouter.ai → **Keys** → **Create Key** (name it e.g. `rag-labs`) → copy it immediately, since it is shown in full only once. The labs use `:free` models; if one is busy or retired, re-run the cell or choose another free model at openrouter.ai/models. Put the key in the lab's `.env` as `OPENROUTER_API_KEY`.

Neo4j Aura Free — Labs 7, 8, 10

1. Go to console.neo4j.io, sign up or log in.
2. Click **Create instance** and choose the **Free** plan (no card needed).
3. Wait a minute or two while it provisions.
4. Open the instance → **Connect**; copy the URI, username and the generated password into `.env`. The password is shown only once; use **Reset password** if you lose it.

Warning: Labs 7, 8 and 10 clear the database (`MATCH (n) DETACH DELETE n`) before they build their graph, so use a throwaway instance, not one that holds data you care about.

Qdrant Cloud Free — Labs 3, 4

1. Go to cloud.qdrant.io, sign up or log in.

2. Click **Create new cluster** and pick the **Free** tier in the nearest region.
3. When it is ready, open **API Keys** → **Create API Key**.
4. Copy the cluster URL (keep **:6333**) and the key into `.env`.

Both labs **recreate their collections on every run**, so use a dedicated cluster or dedicated collection names. **Lab 3 has an alternative with no account**: the optional in-memory Qdrant cell in its Step 3 keeps the vectors in the notebook's memory (they disappear when the kernel restarts). Lab 4 uses the cloud cluster.

PageIndex API key — Lab 9

Sign up at pageindex.ai, open the dashboard's **API Keys** section, create a key, and add it as `PAGEINDEX_API_KEY` next to the OpenRouter key. If the key is missing, the notebook asks for it interactively as a fallback.

Data Setup and Loading

Every notebook loads its data with a **relative path**, which is resolved from the folder Jupyter was launched in. That is why the guide keeps saying “start from inside the lab folder”. If you launch from the repository root or another folder, the notebook either cannot find the file or saves a new one in the wrong place. The labs fall into **three patterns**.

Pattern A — The file ships with the repo and must already be in `data/`

These labs **read** a file and do not download it. You get it by cloning or downloading the whole repository. If you only copied the notebook, the lab fails.

Lab	File	Path the notebook uses	If it is missing
1. Classical RAG	<code>sample_text_document.txt</code>	<code>data/sample_text_document.txt</code>	<code>FileNotFoundError</code> when the file is read
2. Hybrid RAG	<code>sample_text_document.pdf</code>	<code>data/sample_text_document.pdf</code>	The PDF reader fails to open the file
9. Vectorless RAG	<code>CCS-Q1-2025-Earnings-Release.pdf</code>	<code>data/CCS-Q1-2025-Earnings-Release.pdf</code>	<code>FileNotFoundError: Could not find ...</code> Run the notebook from the lab folder

Pattern B — The notebook downloads into a `data/` folder

The notebook creates `data/` if needed and downloads the file(s) from the internet each time you run the download cell. A copy already ships in the repo for most of these, so you usually don't need to do anything, but you do need internet when you run that cell.

Lab	Saved as	Source
5. OCR + RAG	<code>data/debate.pdf</code> , <code>data/invoices/</code> (3 images)	GitHub raw files (debate.pdf, sample invoices and receipts)

Lab	Saved as	Source
6. LLM Wiki	<code>data/SunFactSheet.pdf</code>	NASA Radio JOVE fact sheet. If the NASA host is unreachable, the notebook reuses the copy already in <code>data/</code>
7. Graph RAG	<code>data/document.pdf</code>	arXiv 1706.03762 ("Attention Is All You Need")
8. Graph + Vector	<code>data/downloaded_paper.pdf</code>	arXiv 1706.03762
10. Agentic RAG	<code>data/downloaded_paper.pdf</code>	arXiv 1706.03762

Pattern C — The notebook downloads into the lab folder itself

These two labs do **not** use a `data/` folder. The PDF is saved **directly beside the notebook**, so you will see it appear next to the `.ipynb` file after the download cell runs.

Lab	Saved as	Source
3. Parent-Child Multi-Vector	<code>attention_paper.pdf</code> (in the lab folder)	arXiv 1706.03762
4. ColBERT Late Interaction	<code>bert_paper.pdf</code> (in the lab folder)	arXiv 1810.04805 (BERT)

Files a lab creates for you

Lab 6 writes its generated wiki to an `output_wiki/` folder (an `index.md` plus one page per topic) beside the notebook, and rewrites the index each run. Nothing else needs to be created by hand. Lab 11 (the capstone) has no notebook; it reuses the datasets and tools of the earlier labs.

Quick rules that cover every lab

- **Always start from the lab folder.** Run `cd` into it before `jupyter lab`, or open that folder in VS Code. The same rule applies to `.env` and to `data/`.
- **Clone the whole repo** (or download the ZIP) so the Pattern A files and the bundled copies come with it.
- **Don't rename or move** the files in `data/`; the notebooks refer to them by exact name.
- **Need your own document?** Put it in `data/` and change the path variable in the notebook's load cell (for example `DOC_PATH` in Lab 1 or `PDF_PATH` in Lab 9). Keep the extension the same type the lab expects (Lab 1: text, Labs 2 and 9: PDF).
- **No internet or arXiv blocked?** Download the PDF in your browser, save it with the exact filename and in the exact place from the tables above, and re-run the cells after the download cell.

Checking your data before you run a lab

Paste this into the first cell (or a terminal) from the lab folder. It prints where Jupyter is running and which files it can see:

```

from pathlib import Path
print("Running from:", Path.cwd())
print("data/ contents:", sorted(p.name for p in Path("data").glob("*"))) if
Path("data").exists() else "no data/ folder"
print(".env found:", Path(".env").exists())
print("PDFs in this folder:", sorted(p.name for p in Path(".").glob("*.pdf")))

```

If “Running from” is not the lab folder, close Jupyter, `cd` into the lab folder and start it again (or in Jupyter use **Kernel** → **Restart** after opening the notebook from the right folder).

Lab-by-Lab Cheat Sheet

Lab	Notebook	Main packages installed by its first cell	Keys in .env
1	lab_classical_rag.ipynb	langchain, langchain-openai, langchain-huggingface, sentence-transformers, faiss-cpu	OPENROUTER
2	lab.ipynb	llama-index (core, openrouter LLM, HF embeddings, file readers, BM25), rank_bm25, pymupdf	OPENROUTER
3	lab1_langchain.ipynb	langchain, langchain-qdrant, qdrant-client, pypdf, tiktoken, sentence-transformers	OPENROUTER, QDRANT_URL, QDRANT_API_KEY
4	lab2_colbert.ipynb	qdrant-client, fastembed, langchain-openai, pypdf	OPENROUTER, QDRANT_URL, QDRANT_API_KEY
5	lab_ocr_rag.ipynb	rapidocr, onnxruntime, pymupdf, Pillow, sentence-transformers, faiss-cpu	OPENROUTER
6	llm_wiki.ipynb	PyPDF2, langchain-openai, requests	OPENROUTER
7	graph_rag.ipynb	networkx, neo4j, PyPDF2, yfiles-jupyter-graphs-for-neo4j	OPENROUTER, NEO4J_*
8	vector_graph_hybrid.ipynb	pypdf, neo4j, sentence-transformers, langchain-huggingface, yfiles widget	OPENROUTER, NEO4J_*
9	vectorless_rag.ipynb	pageindex, pymupdf, langchain-openai (also requirements.txt)	OPENROUTER, PAGEINDEX_API_KEY
10	agentic_rag.ipynb	langgraph, langchain-openai, langchain-huggingface, scikit-learn, neo4j, pypdf	OPENROUTER, NEO4J_*

The complete, authoritative install line is always the first code cell of the notebook; this table is only a summary. “NEO4J_*” means `NEO4J_URI` , `NEO4J_USERNAME` , `NEO4J_PASSWORD` (and `NEO4J_DATABASE`).

Troubleshooting Quick Reference

Symptom	Likely cause	Fix
<code>ModuleNotFoundError: No module named 'langchain'</code>	Wrong Python interpreter, venv not activated, or the pip cell was not run	Confirm <code>(.venv)</code> shows in your terminal prompt, re-run the notebook's <code>!pip install</code> cell, or re-select the interpreter in VS Code
<code>'python' is not recognized / 'jupyter' is not recognized</code> (Windows)	Python wasn't added to PATH during install	Re-run the Python installer and check "Add python.exe to PATH", or reinstall from the Microsoft Store
<code>command not found: python3</code> (macOS/Linux)	Python not installed, or only <code>python</code> exists	See Installing Python; try <code>python --version</code> as a fallback
<code>FileNotFoundError: data/...</code> or "Could not find data/..."	Jupyter was not started from the lab folder, or the repo's <code>data/</code> folder is missing	Run the data check cell. <code>cd</code> into the lab folder and restart Jupyter, or clone the full repo (Labs 1, 2, 9 need the bundled files)
A PDF appears in the wrong folder	Notebook was started from another directory	Move the PDF, start Jupyter from the lab folder, re-run. Labs 3 and 4 save the PDF beside the notebook; the others use <code>data/</code>
Download cell fails (arXiv, NASA, GitHub)	No internet, a proxy, or the host is down	Download the file in your browser, save it with the exact filename and location from the data tables, then continue (Lab 6 reuses its bundled copy automatically)
Notebook asks for a key interactively / <code>AuthenticationError</code>	<code>.env</code> missing, misnamed, or in the wrong folder	The file must be exactly <code>.env</code> and sit next to the notebook, not in the repo root; save it and re-run the <code>load_dotenv()</code> cell
Key loads locally but not in Colab	Colab has no persistent disk	Use Colab Secrets, or re-run the <code>%%writefile .env</code> cell each session
<code>401 Unauthorized</code> from OpenRouter	Key copied with a trailing space, quotes, or revoked	Re-copy the key with no quotes or spaces, or create a new one
<code>503 / 504</code> or "Empty Response" from the model	A free model is busy or returned nothing	Re-run the cell; if it persists, choose another <code>:free</code> model at openrouter.ai/models
<code>ConnectTimeout</code> to <code>qdrant.io</code>	The <code>:6333</code> port is missing from <code>QDRANT_URL</code>	Add the port at the end of the URL
Neo4j: "unable to verify that the certificate is valid"	Aura needs TLS	Use the <code>neo4j+s://</code> URI exactly as shown in the Connect dialog
Neo4j auth fails	Wrong password or the instance is paused	Reset the password in the Aura Connect dialog; resume the instance if it is paused
<code>pip install</code> is very slow or hangs	PyTorch and sentence-transformers are large downloads	Wait, or retry with <code>-v</code> ; confirm internet; make sure free disk space is above ~3 GB

Symptom	Likely cause	Fix
First run is slow	Embedding/OCR models download on first use	Normal. Later runs use the cached models
PowerShell: "running scripts is disabled"	Default execution policy blocks venv activation	Run <code>Set-ExecutionPolicy -Scope CurrentUser RemoteSigned</code> once, then retry

© 2026 Cogknowledge Solutions. All rights reserved.